

Dobot CR5AS Manual

Martina

martinaanese05@gmail.com

Contents

1	Introduction	2
2	Specs and data	2
2.1	Specifics of CR5AS	2
2.2	Specifics of DH Robotics PGC-140-50	3
2.3	Connectivity	3
3	Safety protocols	4
4	Connecting to the Dobot	4
4.1	Using the DobotStudio Pro with a PC or tablet	4
4.2	Using the Dashboard and TCP mode	5
4.3	Enabling/Disabling TCP	5
5	How to use DobotStudio Pro	8
6	How to program with Python scripts	16
6.1	Additional information	19



Visit the GitHub repository for the digital version of the manual, example code,
and video tutorials

1 Introduction

I have had the opportunity to use the Dobot CR5AS robot arm at the association Asperastra in Trieste. I had some troubles finding out how to go past the basic features with the proprietary software, so I thought I collected what I learned in a file in order to help anyone who would like to learn to control it.

This manual includes information regarding the Dobot CR5AS robot arm with the DH Robotics PGC-140 gripper.

If you need any help or find any error, please feel free to reach out at martinaanese05@gmail.com.

Have fun!

2 Specs and data

2.1 Specifics of CR5AS

Spec	Value
Payload	5 kg
Working radius	900 mm
Maximum reach	1096 mm
Repeatability	± 0.02 mm
Robot weight	23 kg
Max speed	2 m/s, 223°/s per joint
Collision detection	Present
SafeSkin range	non-contact, 5–15 cm
SafeSkin speed	0.01s detection cycle, 0.1s e-stop execution
SafeSkin material detection	conductive materials only: human body, metals, liquids
SafeSkin joints	J4, J5, J6
Degrees of Freedom	6
Power	100–240V AC in, 48V DC / 20A max out
Environmental constraints	0–45°C, <95% humidity no condensation, IP20

The default password and values for the following elements are:

Element	Value
WiFi name	DobotCR5AS-4520-1795
WiFi password	1234567890
IP address (Ethernet cable)	Specific to each machine
IP address (WiFi connectivity)	Specific to each machine

Element	Value
Administrator mode password	888888

2.2 Specifics of DH Robotics PGC-140-50

Spec	Value
Gripping force	40–140 N
Stroke	50 mm
Payload	3 kg
Repeatability	±0.03 mm
Min opening/closing time	0.6 s
Weight	1 kg
Rated voltage	24 V DC ± 10%
Rated current	0.4 A
Peak current	1 A
Noise emission	< 50 dB
Recommended environment	0–40°C, under 85% RH, IP67
Communication interface	Standard: Modbus RTU (RS485), Digital I/O — Optional: TCP/IP, USB2.0, CAN2.0A, PROFINET, Ether-CAT
Flange mounting	ISO 9409-1-50-4-M6 standard flange

2.3 Connectivity

The CR5AS can communicate with the outside world in two different ways:

- **TCP/UDP, Server or Client:** can be viewed like a phone call. A server is the one sitting by the phone, with a fixed number (IP address + port), waiting for someone to call. A client is the one who dials, it reaches out and starts the connection.

With TCP/UDP, the CR5AS can be set up as either:

- **Server:** Robot waits with a fixed IP; your PC connects to it. Usually used in projects where there is one PC controlling one robot with custom scripts.
- **Client:** Robot initiates the connection out to another device. Often used in industry setups where a central PLC acts as the server and multiple robots connect to it.
- **Modbus, Master or Slave:** Modbus is an older, stricter industrial protocol. The Master is always the one asking questions (“what’s your current position?”, “turn on output 3”) while the Slave only ever answers and can’t initiate questions on its own.

The CR5AS is always the Slave in Modbus, it can never be the Master.

If you're...	Use...
Writing custom control software (Python, C++, your own app)	TCP/IP, robot as Server
Integrating with an existing PLC/SCADA industrial line	Modbus, other device as Master
Building a distributed system with multiple robots reporting to one hub	TCP/IP, robot as Client

A useful example is the Dashboard, which allows the Dobot to be programmable with scripts and using external, non-official tools. With the Dashboard, the Dobot acts as the Server and your script acts as the Client.

The Dashboard is a communication channel (over TCP/IP, on port 29999) that lets a developer send commands to the robot from a script instead of using default elements like a Pendant (physical command center, not always included with the Dobot) or the software DobotStudio Pro.

Before using the Dashboard, the robot must be in TCP mode, which is described in Section 4.3.

The Dobot can either be controlled by scripts in TCP mode or via DobotStudio Pro, which limits the use of third-party or personal elements but is easier to use.

3 Safety protocols

Here are some good safety protocols to use the Dobot safely:

- Always keep the emergency stop button within reach and easily accessible. The e-stop is a physical hardware cutoff: it disconnects power directly, meaning it overrides any code or command currently running, no exceptions. Because it's hardware-level, even a frozen script or a bug in your program can't stop the e-stop from working.
- Keep a clear working space around the robot at all times: no tools, cables, or fragile objects within its reach.
- Stay physically away from the robot's range of motion whenever it's powered on, even if it looks idle.
- Try to always keep SafeSkin active when working near the robot, especially during testing or development. SafeSkin only detects conductive material (skin, metal, liquids); it won't reliably sense non-conductive objects, so don't rely on it as your only safeguard.
- Test new scripts at reduced speed first, especially before running unfamiliar motion sequences.
- If anything behaves unexpectedly (unexpected motion, delayed response, strange sounds), hit the e-stop first, ask questions after.

4 Connecting to the Dobot

4.1 Using the DobotStudio Pro with a PC or tablet

1. Turn the Dobot on, it'll take a couple of minutes to boot up and turn on its WiFi.

2. Download the software from Dobot's Download Center (<https://www.Dobot-robots.com/service/download-center>) and start it.
3. Connect to the WiFi (see name and password in Section 2.1). The Dobot creates its own personal WiFi that you can connect to (or use the Ethernet cable instead). Beware that when connected to its WiFi you will not have internet access.
4. Follow the procedure in Section 4.3 to turn TCP mode off. It only needs to be done the first time you connect after someone else could have turned TCP mode on. If you're the only one using it, you only need to redo this each time you switch modes, not every time you connect to the Dobot. If you skip this or forget, the software will tell you it can't control the robot because it's in TCP mode.
5. In the software, press Connect, then select the robot with the correct IP.

Go to Section 5 to learn about the main elements of the software.

4.2 Using the Dashboard and TCP mode

1. Turn the Dobot on, it'll take a couple of minutes to boot up and turn on its WiFi.
2. Connect to the WiFi (see name and password in Section 2.1). The Dobot creates its own personal WiFi that you can connect to (or use the Ethernet cable instead). Beware that when connected to its WiFi you will not have internet access.
3. Follow the procedure in Section 4.3 to turn TCP mode on. It only needs to be done the first time you connect after someone else could have turned TCP mode off. If you're the only one using it, you only need to redo this each time you switch modes, not every time you connect to the Dobot. If you skip this or forget, you won't be able to program the Dobot freely.

Go to Section 6 to learn about TCP programming.

4.3 Enabling/Disabling TCP

I have added a video of the procedure to disable TCP (enabling it follows the same structure) in the repository this manual is attached to.

You need to enable TCP if you want to program it with custom Python scripts, and turn it off if you'd like to program it with the DobotStudio Pro software.

Enabling TCP

1. Turn on the Dobot, wait for it to boot completely and connect to its WiFi. Open the terminal.

The following code blocks include the example IP `192.168.201.1` and you will have to swap it out with your Dobot's IP.

For both Mac and Windows users:

2. Check the robot's connection:

```
ping 192.168.201.1
```

On Mac, after a while you have to interrupt it with Control+C.

If run correctly, you'll get something like:

```

PING 192.168.201.1 (192.168.201.1): 56 data bytes
64 bytes from 192.168.201.1: icmp_seq=0 ttl=64 time=52.600 ms
64 bytes from 192.168.201.1: icmp_seq=1 ttl=64 time=89.080 ms
64 bytes from 192.168.201.1: icmp_seq=2 ttl=64 time=112.341 ms
64 bytes from 192.168.201.1: icmp_seq=3 ttl=64 time=139.299 ms
64 bytes from 192.168.201.1: icmp_seq=4 ttl=64 time=154.528 ms
64 bytes from 192.168.201.1: icmp_seq=5 ttl=64 time=55.060 ms
64 bytes from 192.168.201.1: icmp_seq=6 ttl=64 time=23.080 ms
^C
--- 192.168.201.1 ping statistics ---
7 packets transmitted, 7 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 23.080/89.427/154.528/45.023 ms

```

Figure 1: Expected output of a successful ping to the robot.

If the packet loss is close to 0%, you can continue. Otherwise, retry the ping or check that you are correctly connected to the Dobot's WiFi.

3. Connect via SSH:

```
ssh root@192.168.201.1
```

This step may ask for a password, it's usually "dobot".

4. Backup the current settings:

```
cp /dobot/userdata/project/settings/function/remoteControl.json /
dobot/userdata/project/settings/function/remoteControl.json.bak
```

5. Set TCP mode:

```
echo '{"mode": "tcp"}' > /dobot/userdata/project/settings/function/
remoteControl.json
echo '{"mode": "tcp"}' > /dobot/userdata/user_project/settings/
function/remoteControl.json
```

6. Enable the remote switch:

```
echo '{"value": true}' > /dobot/userdata/project/settings/function
/remoteSwitch.json
echo '{"value": true}' > /dobot/userdata/user_project/settings/
function/remoteSwitch.json
```

7. Reboot:

```
reboot
```

8. Then turn the Dobot off by pressing and holding the on/off button for 4 seconds, then wait for 30s and turn it back on by pressing the on/off button.

Disabling TCP mode

1. Turn on the Dobot, wait for it to boot completely and connect to its WiFi. Open the terminal.

The following code blocks include the example IP `192.168.201.1` and you will have to swap it out with your Dobot's IP.

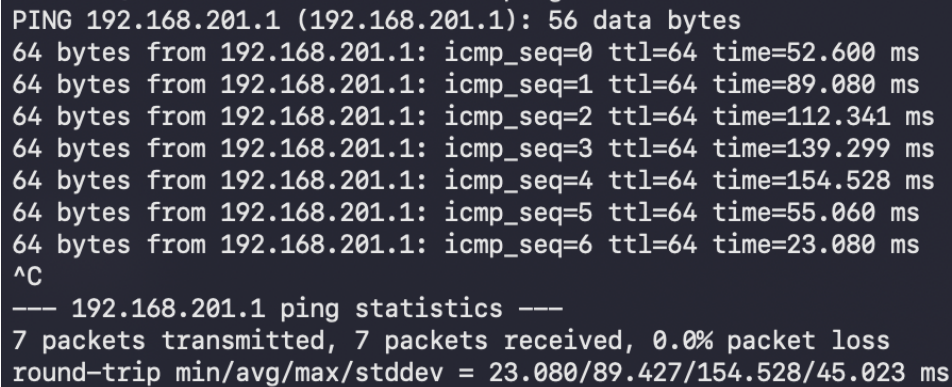
For both Mac and Windows users:

2. Check the robot's connection:

```
ping 192.168.201.1
```

On Mac, after a while you have to interrupt it with Control+C.

If run correctly, you'll get something like:



```
PING 192.168.201.1 (192.168.201.1): 56 data bytes
64 bytes from 192.168.201.1: icmp_seq=0 ttl=64 time=52.600 ms
64 bytes from 192.168.201.1: icmp_seq=1 ttl=64 time=89.080 ms
64 bytes from 192.168.201.1: icmp_seq=2 ttl=64 time=112.341 ms
64 bytes from 192.168.201.1: icmp_seq=3 ttl=64 time=139.299 ms
64 bytes from 192.168.201.1: icmp_seq=4 ttl=64 time=154.528 ms
64 bytes from 192.168.201.1: icmp_seq=5 ttl=64 time=55.060 ms
64 bytes from 192.168.201.1: icmp_seq=6 ttl=64 time=23.080 ms
^C
--- 192.168.201.1 ping statistics ---
7 packets transmitted, 7 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 23.080/89.427/154.528/45.023 ms
```

Figure 2: Expected output of a successful ping to the robot.

If the packet loss is close to 0%, you can continue. Otherwise, retry the ping or check that you are correctly connected to the Dobot's WiFi.

3. Connect via SSH:

```
ssh root@192.168.201.1
```

This step may ask for a password, it's usually "dobot".

4. Backup the current settings:

```
cp /dobot/userdata/project/settings/function/remoteControl.json /
dobot/userdata/project/settings/function/remoteControl.json.bak
```

5. Set TP mode:

```
echo '{"mode": "tp"}' > /dobot/userdata/project/settings/function/
remoteControl.json
echo '{"mode": "tp"}' > /dobot/userdata/user_project/settings/
function/remoteControl.json
```

6. Disable the remote switch:

```
echo '{"value": false}' > /dobot/userdata/project/settings/
function/remoteSwitch.json
echo '{"value": false}' > /dobot/userdata/user_project/settings/
function/remoteSwitch.json
```

7. Reboot:

```
reboot
```

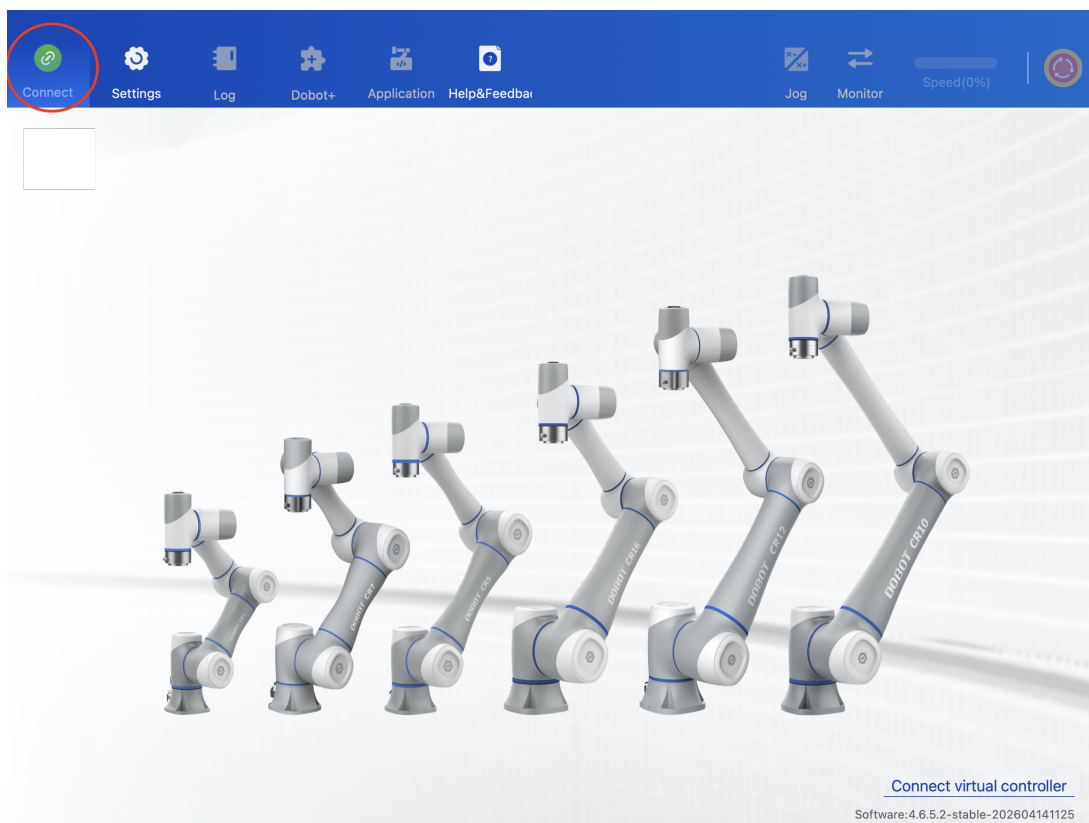
- Then turn the Dobot off by pressing and holding the on/off button for 4 seconds, then wait for 30s and turn it back on by pressing the on/off button.

5 How to use DobotStudio Pro

DobotStudio Pro is the original software to control and program any Dobot robotic arm.

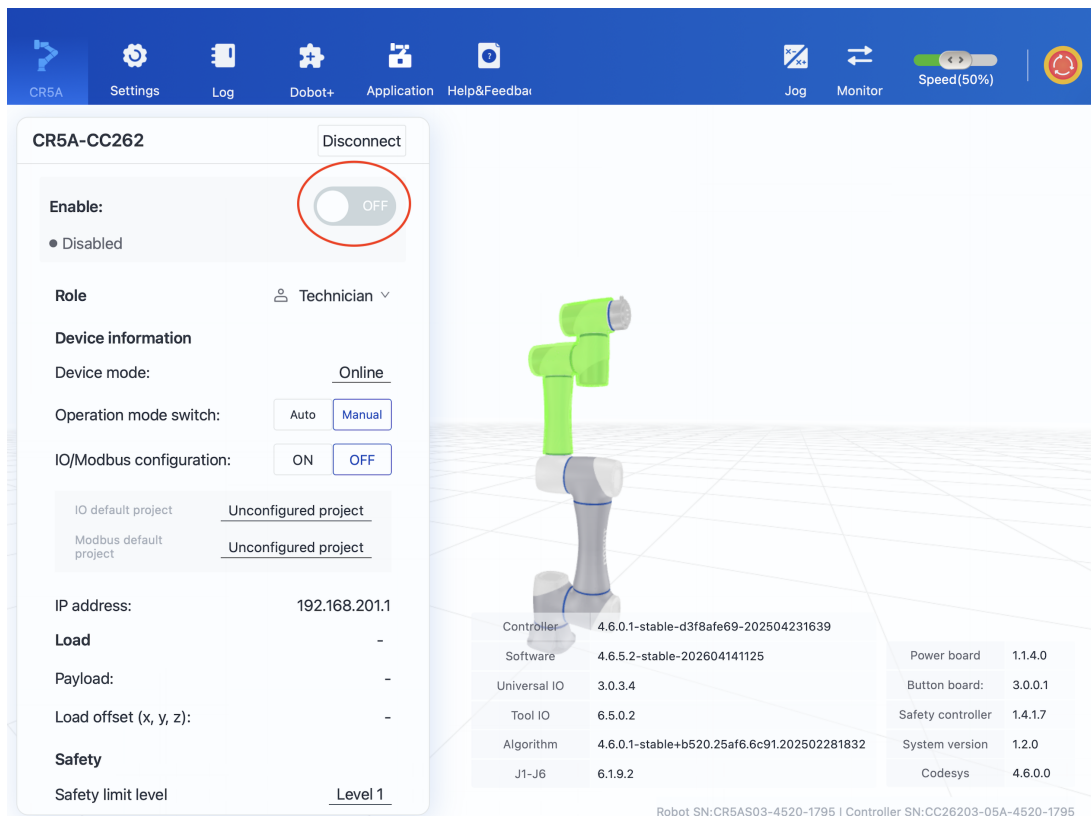
Following is a brief explanation of the main features of the software, but I recommend taking a look at the file called “DobotStudio Pro User Guide (CR&Nova) V2.8.0”, which you can download at <https://www.Dobot-robots.com/service/download-center>, or taking a look at the site https://docs.trossenrobotics.com/Dobot_cr_cobots_docs/Dobotstudiopro/programming.html, where they go deeper into it.

- This is the page the software opens on. With “Connect”, you’ll be able to select your Dobot by its IP and connect to it. Make sure you’re connected to the Dobot’s WiFi.

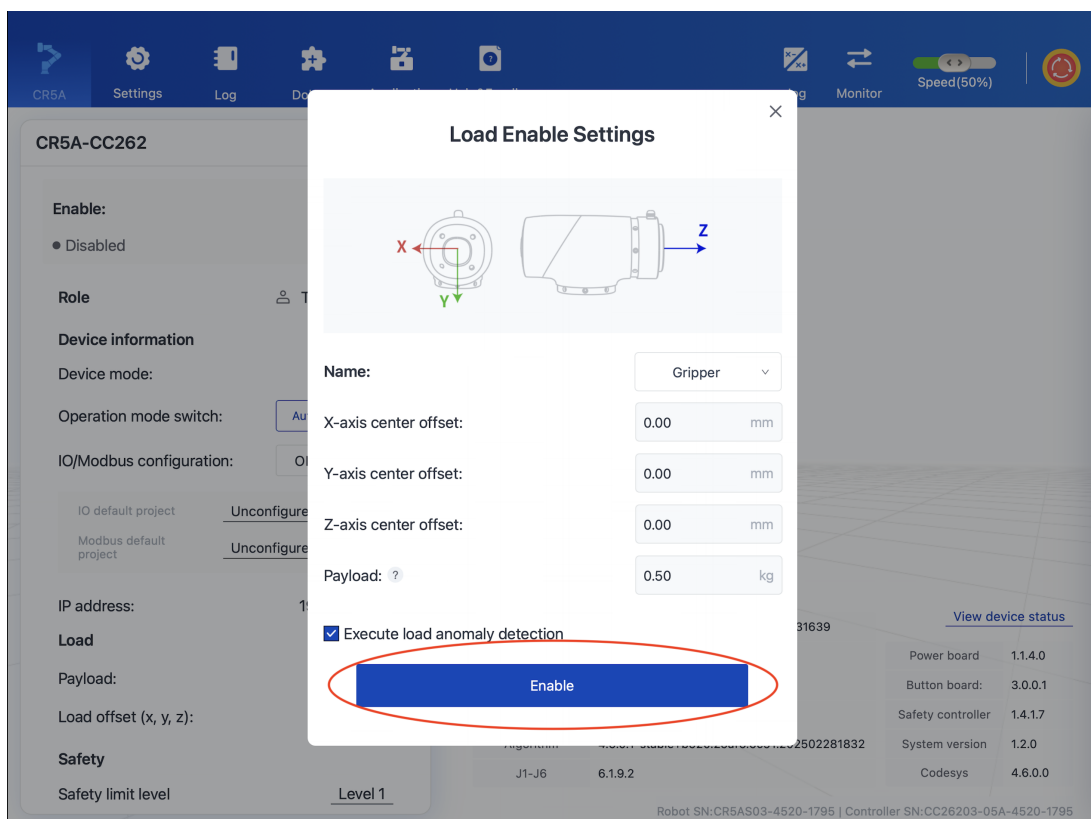


- By pressing “Enable”, you’re enabling the Dobot to be controlled. Note that on the right side of the screen you’re able to see the Dobot’s position at any time and, if SafeSkin is on, to see if parts of it detect a close object.

The color of the little Dobot icon in the top left corner is indicative of its status: blue means disabled, green means enabled and working correctly, yellow means there is a non-threatening error (such as a collision with the SafeSkin), and red means a threatening error.

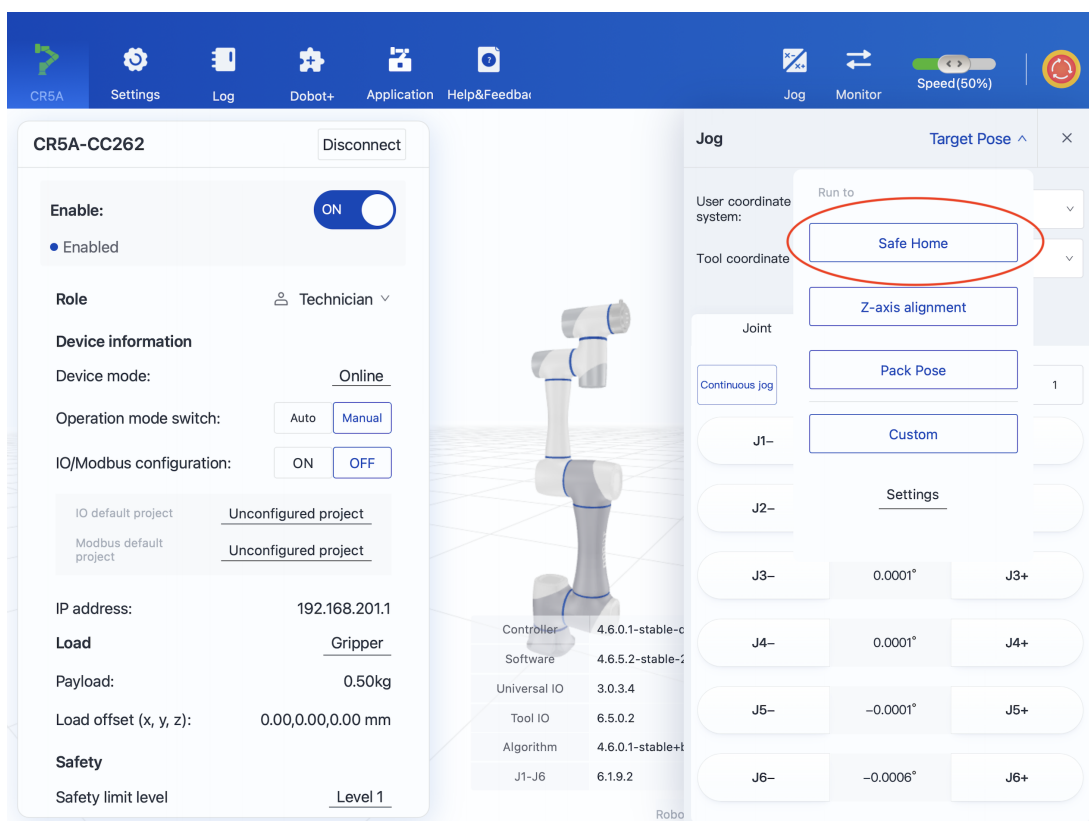
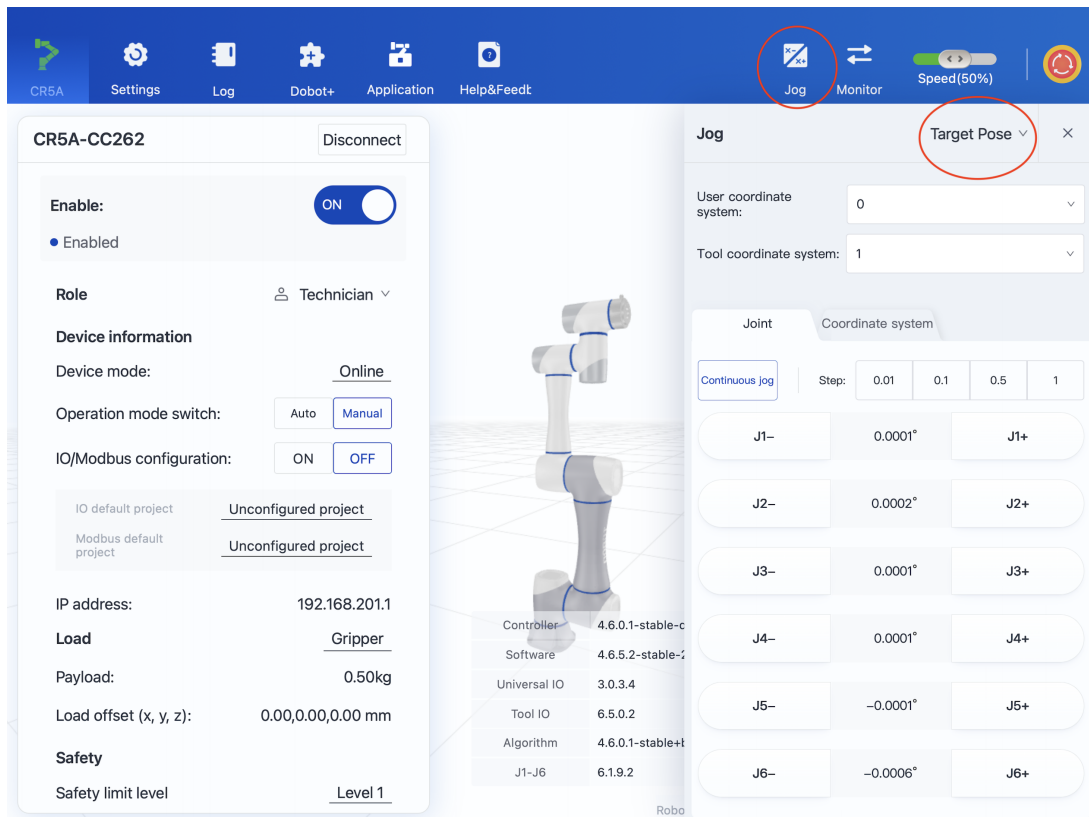


- Once you press Enable, the Dobot will perform a Load examination. If the numbers are correct, confirm Enable.

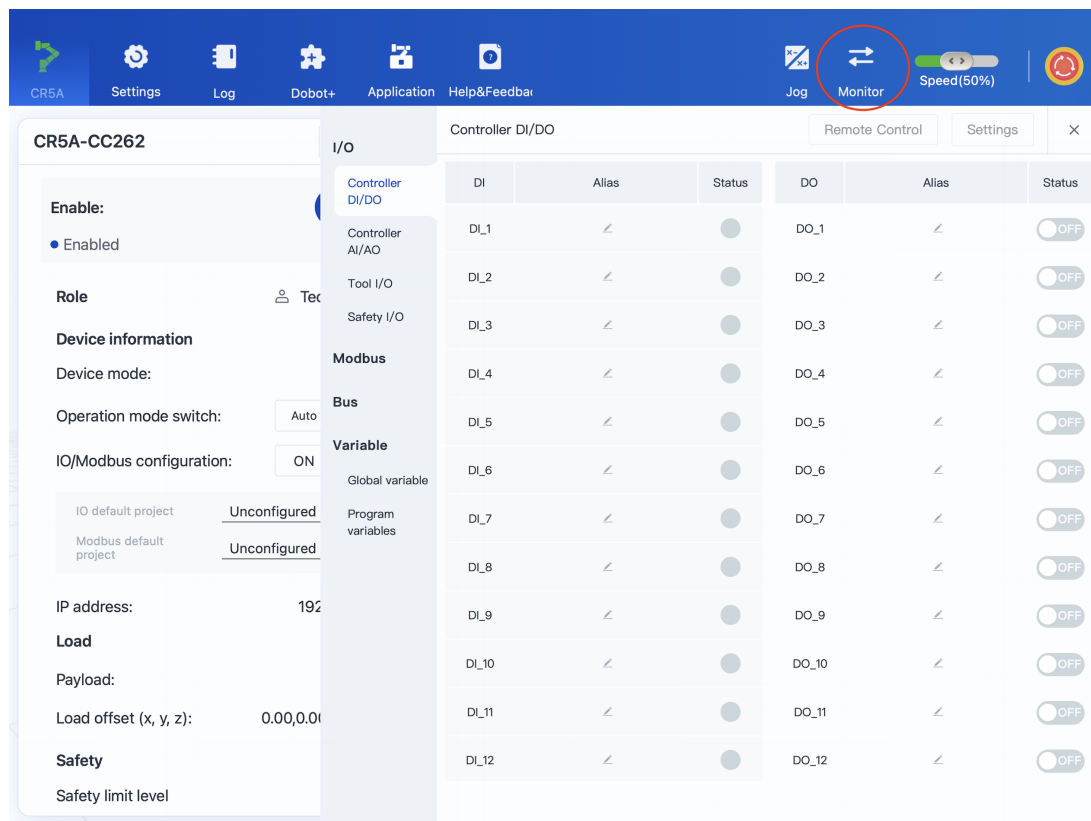


- By selecting “Jog” you are able to move the Dobot one joint at a time. This option also

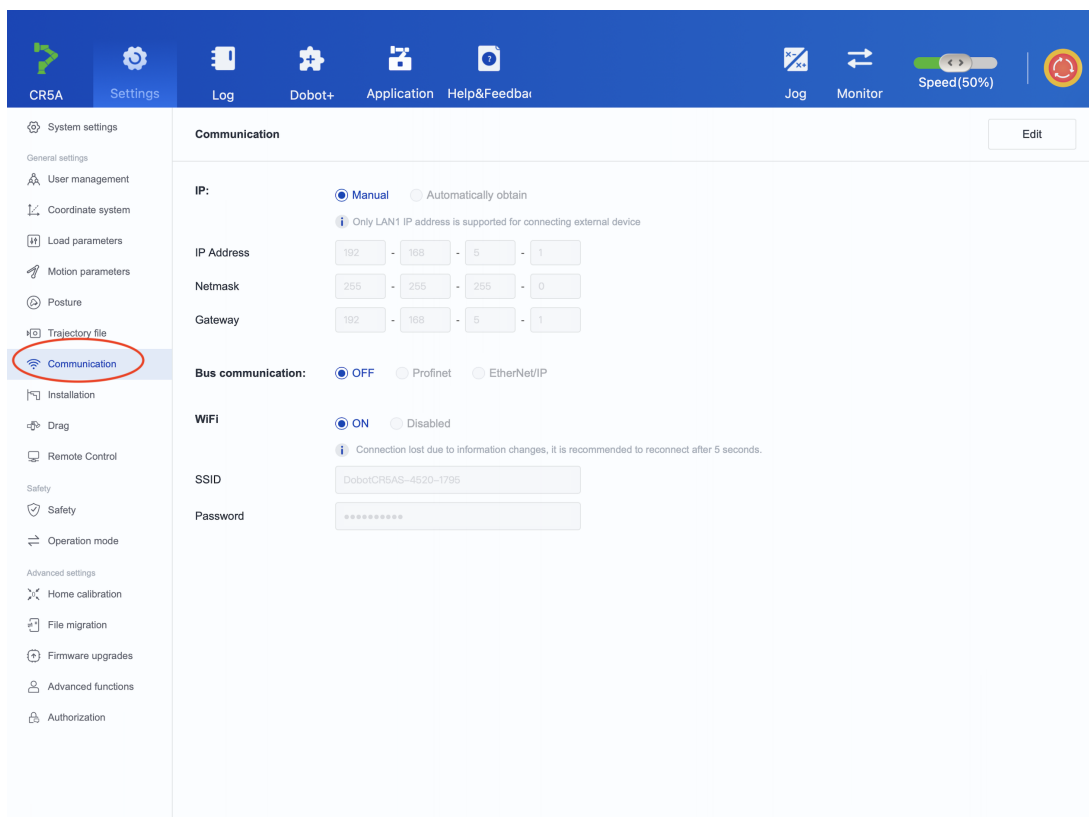
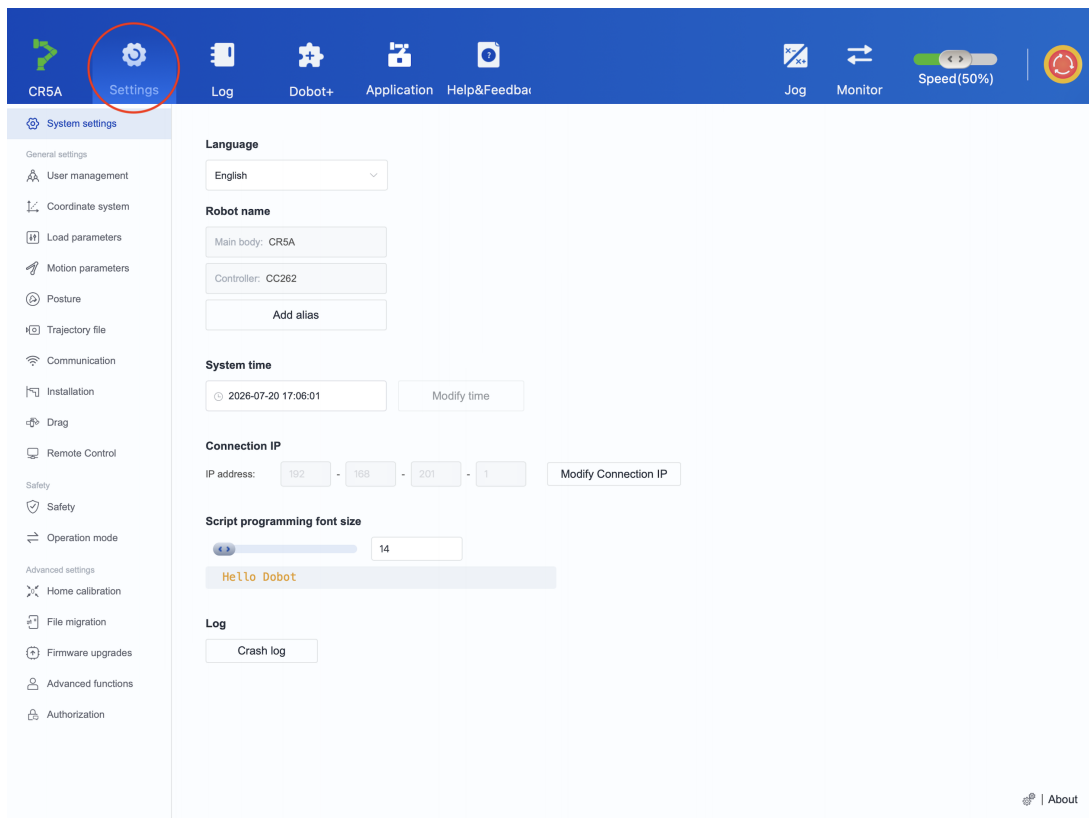
contains “Target Pose”, with which you’ll be able to gradually move the Dobot to some predefined positions like Safe Home and Pack Pose.



- By selecting “Monitor” you’ll be able to dive into more functions related to Input/Output.



- This is the Settings panel, which allows you to tweak the Dobot. An important setting to note is that by going into “Communications” you’ll be able to view and edit the Dobot’s IP address.



- By pressing “Dobot+” you’ll be able to explore the extensions; some important ones are the Gripper and SafeSkin extensions.

CR5A

Settings

Log

Dobot+

Application

Help&Feedba

Jog

Monitor

Speed(50%)

Dobot+

Import

Uninstall

End shortcut key

	DHGrip_v2-1-6-stable	Joint type adaptive electric claw, plug and play with cooperative robot	Open
	SafeSkin_v2-0-3-stable	SafeSkin plugin	Open
	DobotES01_v1-0-3-stable	DobotES suckers, plug and play with cobots	Install
	RobotiqEpick_v2-0-5-stable	Robotiq series gripper	Install
	OnRobot_v2-1-2-stable	OnRobot plugin, compatible with 2FG7 gripper, RG series grippers, and VG series suction cups.	Install
	EndButtonSetting_v1-0-0-stable	Allows setting the end ecological button to the save point function	Install
	ForceTorqueSensor_v3-4-2-stable	ForceTorqueSensor plug-in	Install
	RobotiqGrip_v2-1-5-stable	Robotiq series gripper	Install
	Enlight v1-0-0-stable	Enlight v1-0-0-stable	Install

[Plugins and manual downloads](#)

CR5A

Settings

Log

Dobot+

Application

Help&Feedba

Jog

Monitor

Speed(50%)

Dobot+

DHGrip_v2-1-6-stable

Status

Connected

Init

Control

Current width

100

%

Target width

-

100

+

0%

100%

Force

-

100

+

0%

100%

Speed

-

100

+

0%

100%

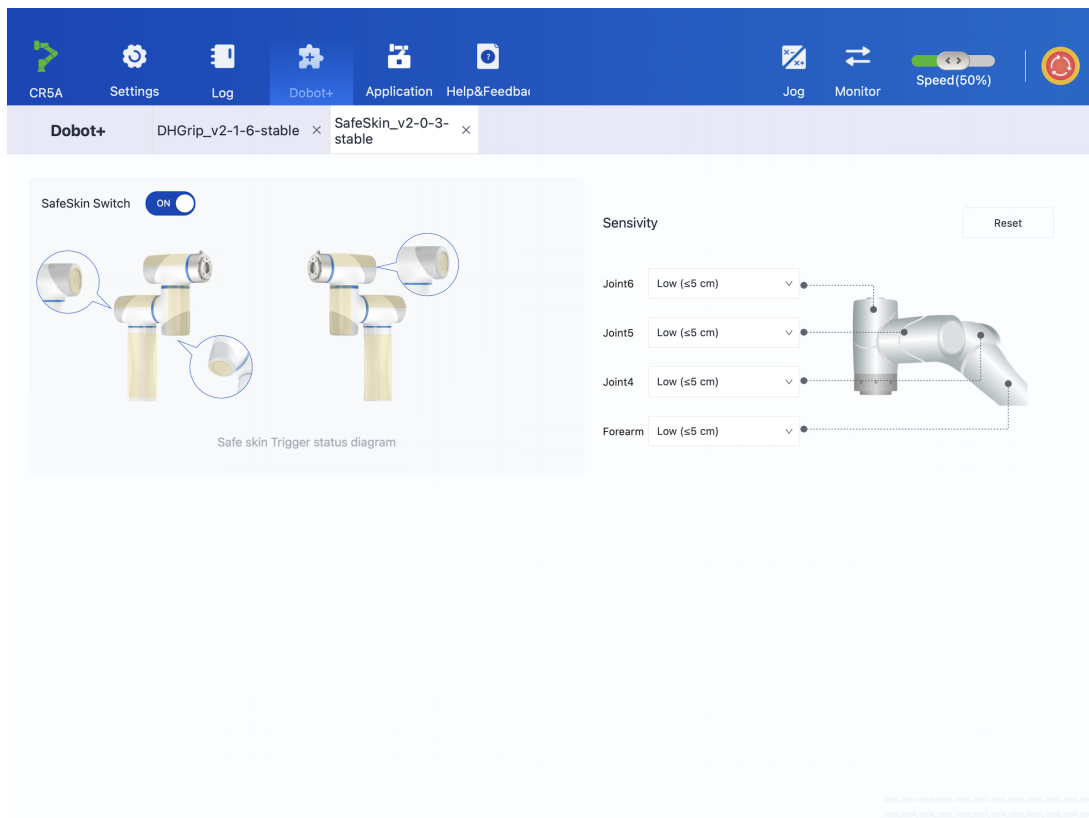
Setting

ID

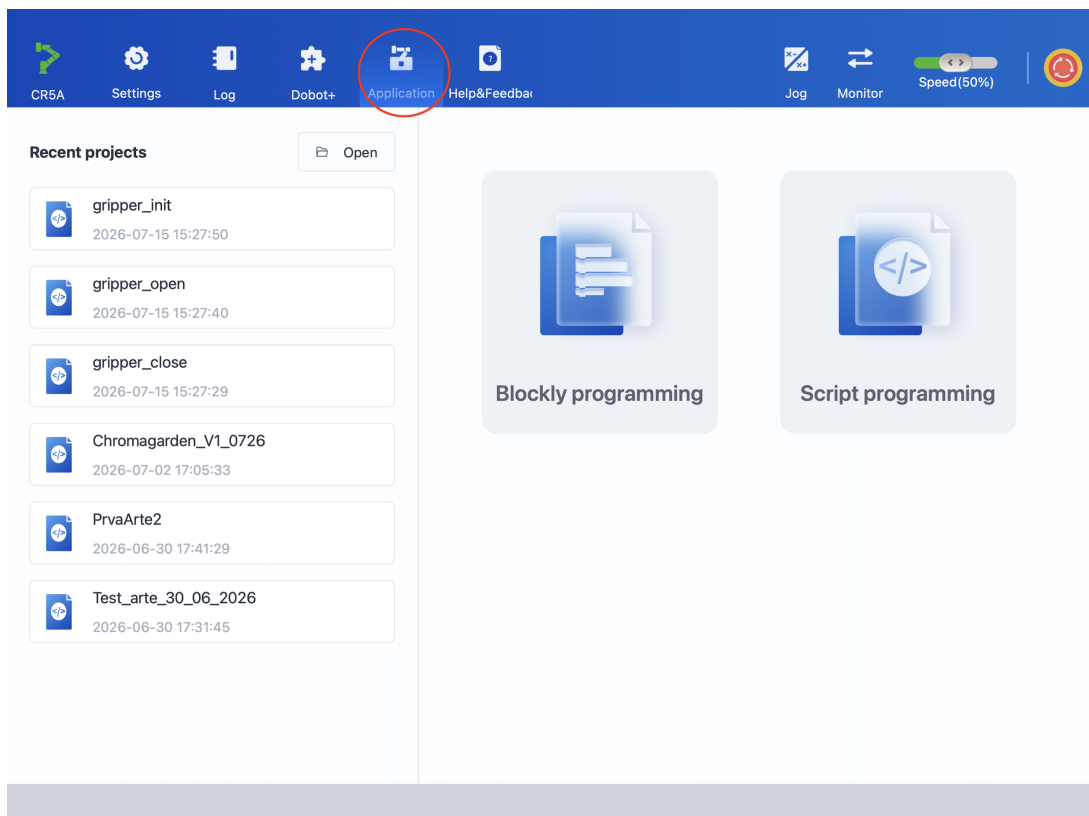
1

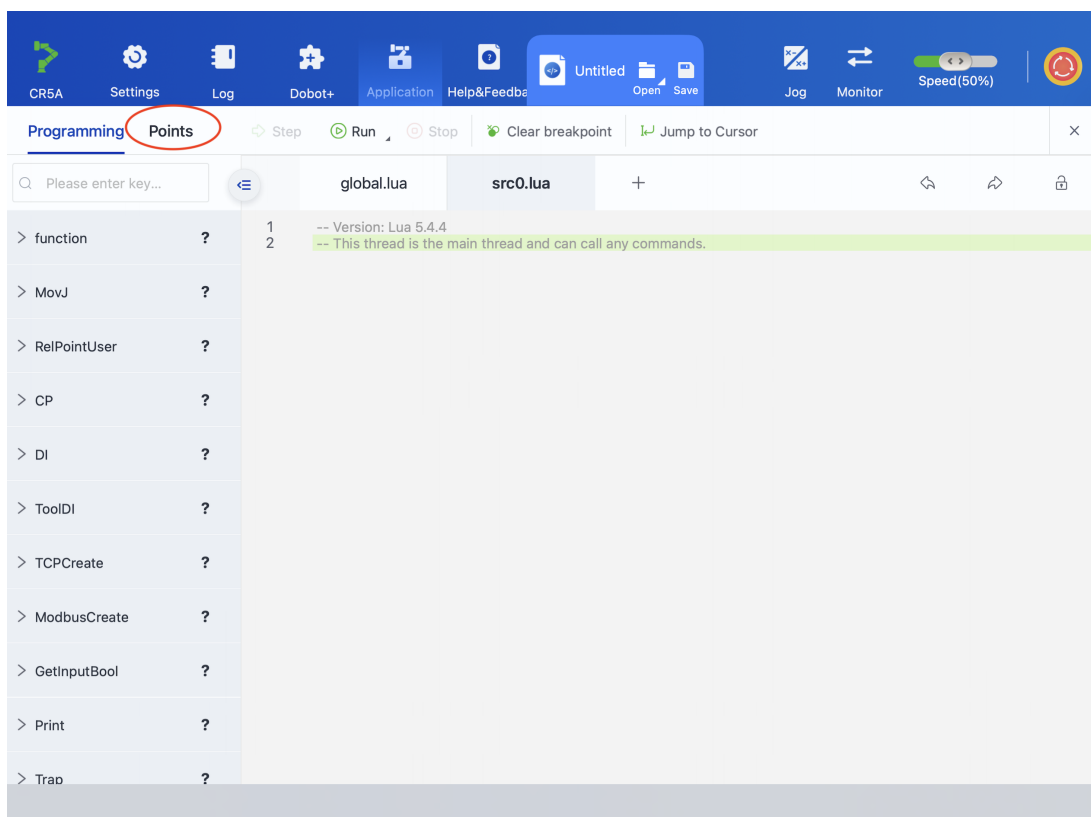
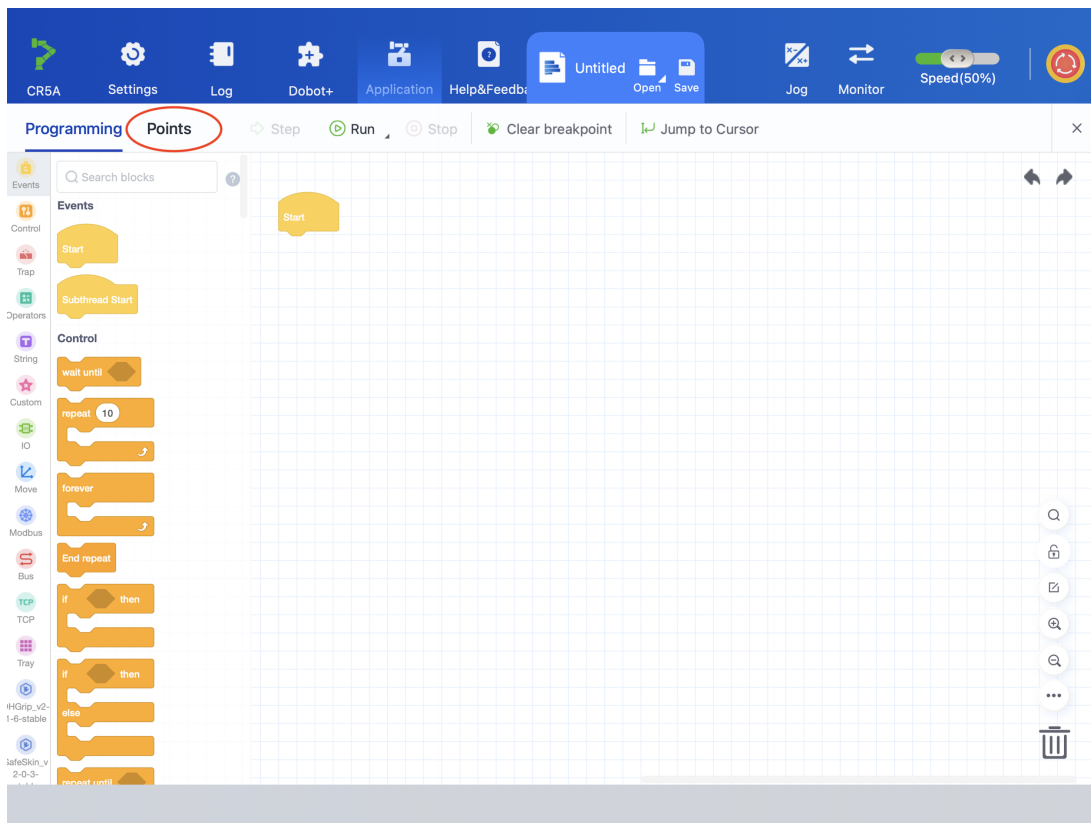
BaudRate

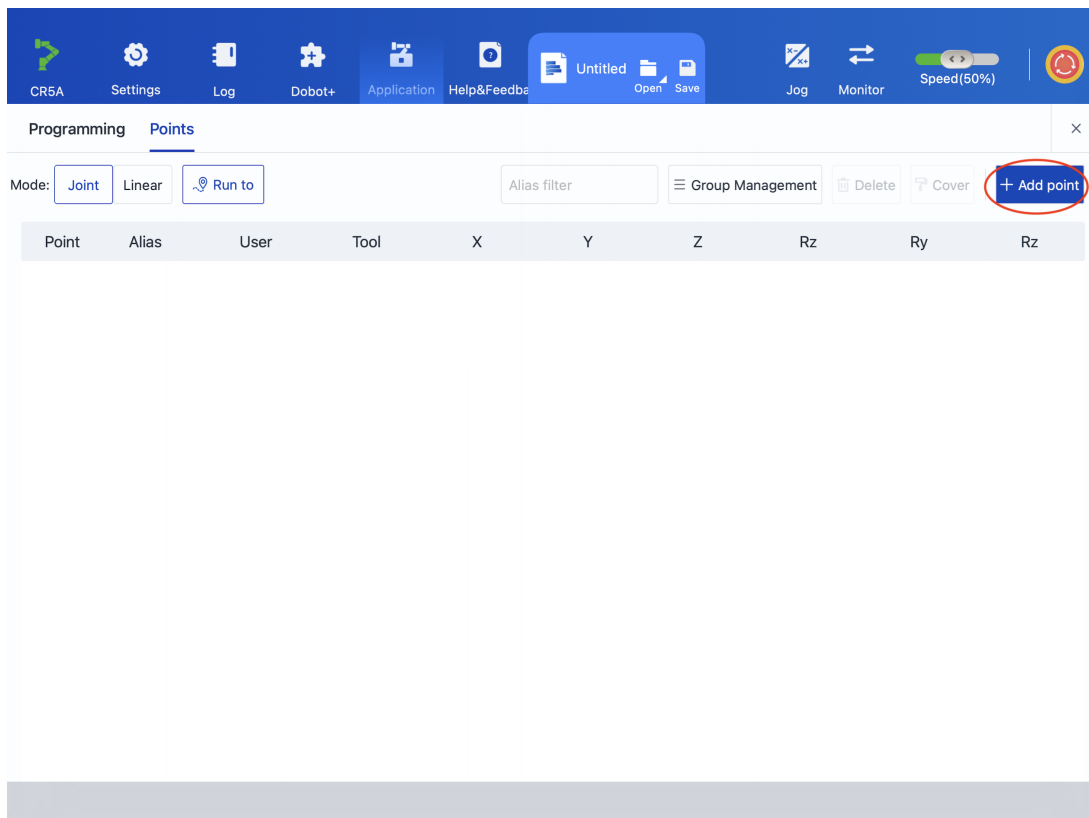
115200



- By pressing “Application” you’ll be able to program the Dobot using the software’s functions. You have the option to do Blockly programming and Script programming. Both are valid and have the option “Points”, which allows you to record points while in Drag Mode and use them in your script.

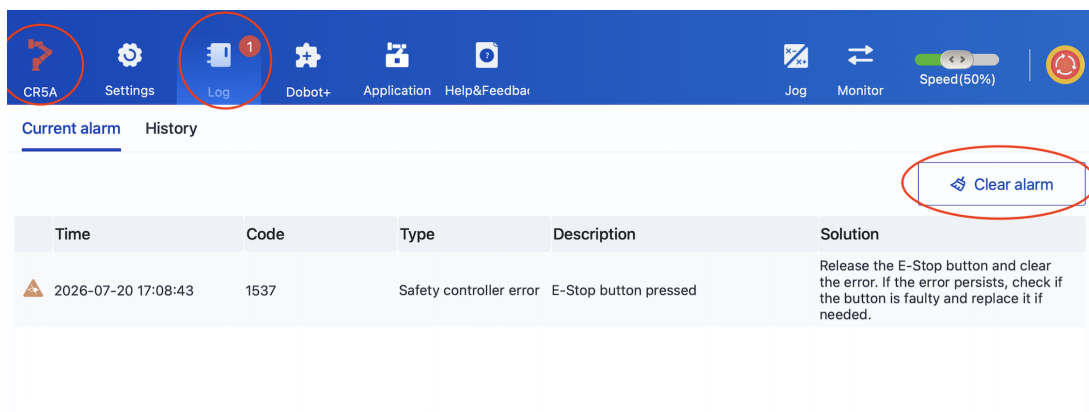






- Whenever the E-Stop button is pressed, or any other error occurs, you can view and clear these from the “Log” panel.

Please note that, whenever the E-Stop button is pressed, you’ll have to disengage it, clear the alarm, and then go back to the “CR5A” panel to Enable the Dobot again.



6 How to program with Python scripts

Following is a brief explanation of the main features and commands that allow you to control the Dobot via scripts in TCP mode.

The library I decided to use is the official one; it’s available on GitHub at <https://github.com/Dobot-Arm/TCP-IP-Python-V4>, but I have also put it into the repository this manual is attached to.

Here is a list of all the commands that could be helpful:

Connection / power state

- `EnableRobot()` — powers up the arm
- `DisableRobot()` — powers it down
- `ClearError()` — resets alarm state

Motion

- `MovJ()` — joint-space point-to-point move
- `MovL()` — linear (Cartesian) move
- `StartDrag()` — enters drag/teach mode (manual hand-guiding)
- `StopDrag()` — exits drag mode
- `SpeedFactor()` — sets global speed ratio

Status / feedback

- `RobotMode()` — returns current robot state (enabled, running, error, etc.)
- `GetAngle()` — reads current joint angles
- `GetPose()` — reads current Cartesian position

Kinematics

- `InverseKin()` — converts a Cartesian target into joint angles

Gripper / tool I/O

- `SetToolPower()` — turns tool-side power on/off (used before gripper control)
- `ToolDOInstant()` — toggles a tool digital output instantly, used to open/close the gripper
- `GetToolDO()` — reads current tool digital output state

Modbus (for grippers that use it, like DH Robotics PGC-140-50)

- `ModbusRTUCreate()` — opens a Modbus RTU connection to the gripper (slave ID, baud rate, parity, data bits, stop bits)
- `ModbusClose()` — closes that Modbus connection

To control a DH gripper you can either use the Modbus protocol (it's not advised, since you'd have to have the gripper connected to the Dobot's main computer with a cable that is not always included) or use the Remote Control protocol, which has the Python code run prewritten scripts in DobotStudio Pro. The latter is the best option and only requires an initial setup that can be cumbersome, but is much easier to use afterward.

Remote control for the DH Gripper

Setup:

1. Disable TCP mode.
2. Connect the Dobot to DobotStudio Pro.

3. Download the DH gripper plug-in from the list provided in the software.
4. In the programming section of the software, add the 3 main scripts (they are included in the “Examples” folder, in a folder that specifies those are the Lua scripts connected to Example 11, which moves the gripper).
5. Enable TCP mode. You will now be able to activate these 3 scripts as needed (see the “Examples” folder for the code structure). The only caveat is that you need to have the DobotStudio Pro software always open in the background, otherwise it will not work properly.

I have added an example file in the GitHub repository for each of these, in the folder “Examples”. To run these correctly, download the “Examples” folder, unzip it, and run the file you’re interested in. The Dobot library was already included in this folder for easy use. With other libraries, you can just download them onto your PC and don’t have to reference them every time, but this library doesn’t have a `setup.py` file, so that can’t be done here.

Example file	What it does
<code>00_hello_world.py</code>	Connects, enables the robot, and moves J1 back and forth from 0° to $\pm 30^\circ$ in order to show the basic connect $\rightarrow$ enable $\rightarrow$ move $\rightarrow$ disable pattern.
<code>01_connect_and_status.py</code>	Connects, clears errors, enables the robot, and reads its current status via <code>RobotMode()</code> .
<code>02_joint_motion.py</code>	Demonstrates <code>MovJ()</code> joint-space motion, moving J1 to 30° and reading joint angles before/after with <code>GetAngle()</code> .
<code>03_linear_motion_and_pose.py</code>	Demonstrates <code>MovL()</code> straight-line motion, lifting the tool 50mm in Z using Cartesian coordinates read via <code>GetPose()</code> .
<code>04_drag_mode.py</code>	Puts the robot into hand-guided drag mode (<code>StartDrag()</code> / <code>StopDrag()</code>) so you can move it manually and read the resulting position.
<code>05_inverse_kinematics.py</code>	Uses <code>InverseKin()</code> to calculate what joint angles a target X/Y/Z position would require, without actually moving there.
<code>06_drag_record_and_replay.py</code>	Lets you drag the robot to multiple positions, records each one’s joint angles on Enter, then replays the whole sequence with <code>MovJ()</code> .
<code>07_interactive_joint_jog.py</code>	An interactive loop that asks which joint to move and to what angle, then moves only that joint.
<code>08_record_cartesian_points.py</code>	Same idea as file o6, but records/replays Cartesian X/Y/Z positions (<code>GetPose()</code> / <code>MovL()</code>) instead of joint angles.
<code>09_closest_reachable_position.py</code>	Tries to reach a user-given X/Y/Z target, and if unreachable, backs off step-by-step (Z, then Y, then X) until it finds a reachable point.
<code>10_speed_comparison_test.py</code>	Repeats the same J1 move ($0^\circ \rightarrow 180^\circ$) at five different <code>SpeedFactor()</code> settings so you can compare how speed affects motion.

Example file	What it does
<code>11_gripper_init_open_close.py</code>	Triggers the gripper's init/close/open Lua scripts over a raw socket and re-arms the robot (<code>ClearError()</code> + <code>EnableRobot()</code>) after each one.

6.1 Additional information

- While using the DobotStudio Pro software, you may enable Drag Mode by pressing the physical button on J6 of the Dobot for 4 seconds, until the LED starts to flash purple, and disable it by pressing it once. If SafeSkin is enabled, which is recommended to always be the case, it might give you an error while trying to reach for the button. You can avoid this by temporarily lowering the SafeSkin precision in the settings, but remember to set it back to normal when you're done with Drag Mode.
- When using Inverse Kinematics, both in TCP and TP mode, you might often get an error that says the position is not reachable, even though you might have been able to position the robot there with Drag Mode. This is due to the mathematical calculations behind this function, which are very precise and might be able to reach a point really close to the one you selected, off by some millimeters (which in Drag Mode you don't notice), but not that specific point. You might solve this problem by allowing the Dobot to reach the closest available point to the one you selected, or by adding another Inverse Kinematics library.